CS0054
(INTELLIGENT SYSTEMS)

EXERCISE

4

NATURAL LANGUAGE PROCESSING (NLP)

Group Name:	Christian Dave K. Salvador
Members:	**Name** / **Role**
Section:	AN42
Professor:	Ms. Crisola G. Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: E]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design Intelligent system based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Know and understand natural language processing & language technology.
- Familiarize with the approaches to NLP.
- Learn sample applications of NLP.

IV. BACKGROUND INFORMATION

What is NLP?

Working in Data Science and having a background in Technical Writing, I was drawn to the field of Natural Language Processing (NLP). Machines understanding language fascinates me, and I often ponder which algorithms Aristotle would have used to build a rhetorical analysis machine if he had the chance. If you're new to Data Science, getting into NLP can seem complicated, especially since there have been so many recent advancements in the field. It is hard to know where to start.

What can Machines Understand?

While a computer can be actually quite good at finding patterns and summarizing documents, it must transform words into numbers before making sense of them. This transformation is needed because machines "learn" thanks to mathematics, and math doesn't work very well on words. Before transforming the words into numbers, they are often cleaned of things like special characters and punctuation, and modified into forms that make them more uniform and interpretable.

What are Word Clouds?

Word clouds (also known as text clouds or tag clouds) work in a simple way: the more a specific word appears in a source of textual data (such as a speech, blog post, or database), the bigger and bolder it appears in the word cloud.

A word cloud is a collection, or cluster, of words depicted in different sizes. The bigger and bolder the word appears, the more often it's mentioned within a given text and the more important it is.

Also known as tag clouds or text clouds, these are ideal ways to pull out the most pertinent parts of textual data, from blog posts to databases. They can also help business users compare two different pieces of text to find the wording similarities between the two.

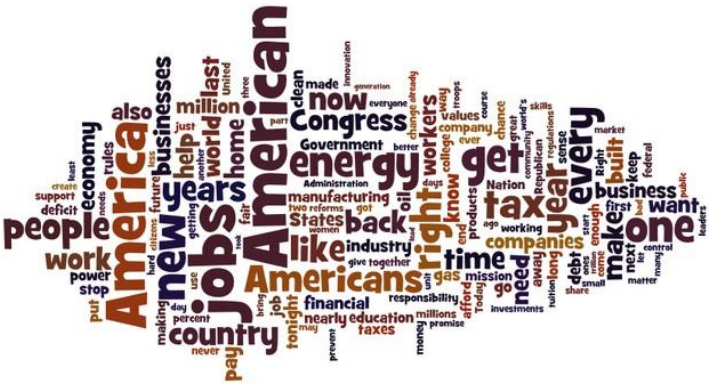
For instance, you might ask current customers what they like or don't like about your new product line. Or, you could ask them to give suggestions on how your organization could improve. They could also have the chance to elaborate on any pain points they're experiencing.

There are industry tools that allow you to code such open-ended data so users can understand it quantitatively. Yet, these don't come cheap. Word clouds offer a cost-effective, yet powerful, alternative.

With these, you can still quantify your text-based insights into measurable analytics. The only difference? You won't create a chart or graph as you would with a set of numbers.

Instead, you'll create a word cloud generator to transform the most critical information into a word cloud.

Here's an example from USA Today using U.S. President Barack Obama's State of the Union Speech 2012:



As you can see, words like "American," "jobs," "energy" and "every" stand out since they were used more frequently in the original text.

Now, compare that to the 2014 State of the Union address:



V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY (50 POINTS)

Data Set:

- **Spam Email Data Set found here:**
  https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb

Steps:

1. Importing Dependencies and Data

Start by importing the dependencies and the data. The data is stored as a comma separated values (csv) file, so use pandas' read_csv() function to open it into a DataFrame.

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud


#create dataframe from csv
df = pd.read_csv('emails.csv')


df.head()
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

df.head()

2. Exploratory Analysis

Before anything, it is best to do a quick analysis of the data to eliminate duplicate rows and establish some baseline counts. Use pandas drop_duplicates to drop the duplicate rows.

```
print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
df = df.reset_index(inplace = False)[['text','spam']]


print(df.shape)
```

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```

Counts and shape before/after duplicate removal

3. Cleaning the Texts

Notice in the word cloud image, all the text is lower case. There are no punctuation marks or special characters. That's because the text has been cleaned to make it easier to analyze. Using regular expressions, it is easy to clean the text using a loop:

```
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)


#assign the cleaned descriptions to the data frame
df['text'] = clean_desc

df.head(3)
```

	text	spam
0	subject naturally irresistible your corporate ...	1
1	subject the stock trading gunslinger fanny is ...	1
2	subject unbelievable new homes made easy im wa...	1

4. Removing Stop Words

Stop words are the most common words like “the” and “of.” Removing them from the email text allows the more relevant frequent words to stand out. Removing stop words is a common technique! Some Python libraries like NLTK come pre-loaded with a list of stop words, but it is easy to create one from scratch.

```
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']
```

5. Construct the Word Cloud

Conveniently, there is a Python library for creating word clouds. It can be installed using pip.

```
pip install wordcloud
```

When constructing the word cloud, it is possible to set several parameters like height and width, stop words, and max words. It is even possible to shape it instead of displaying the default rectangle.

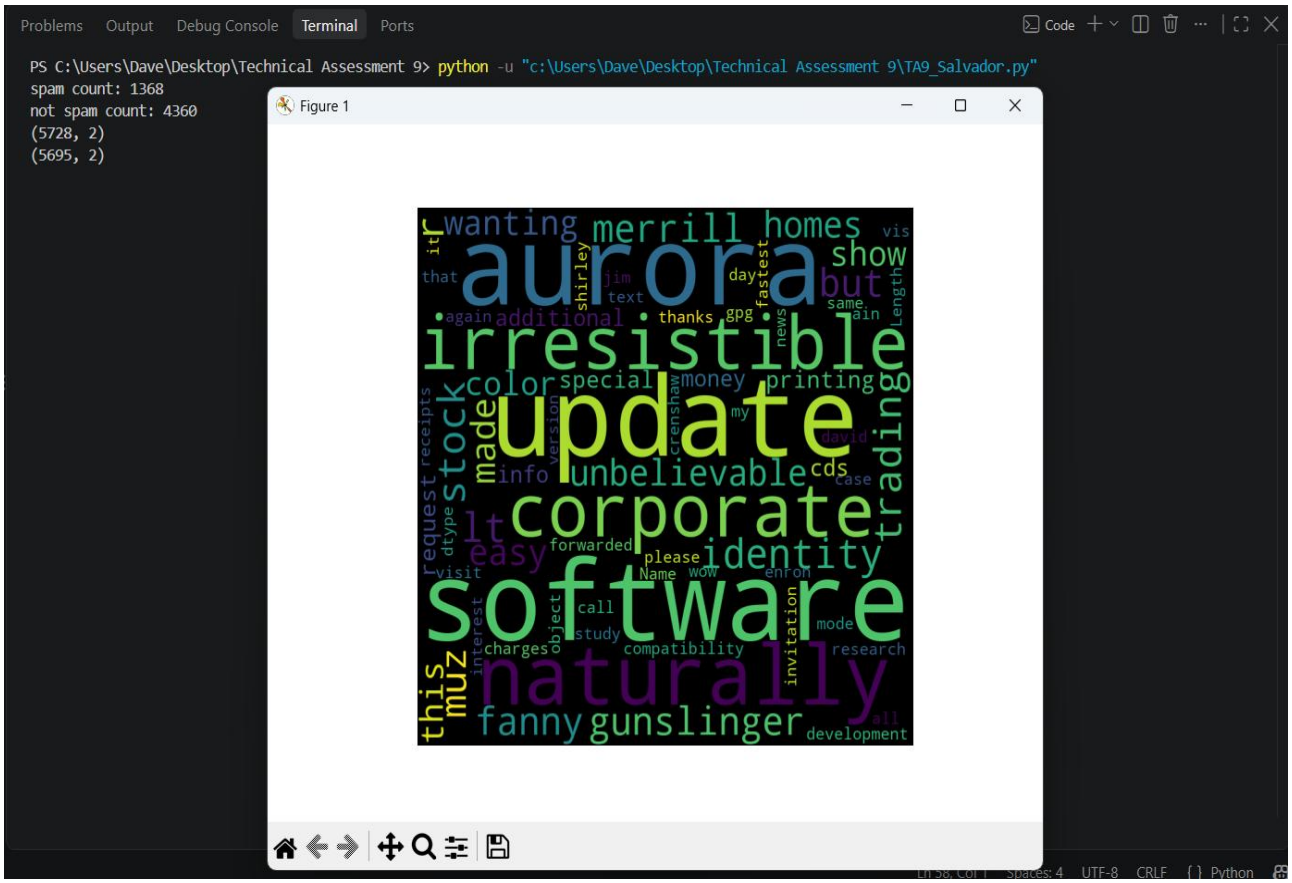
```
wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df1['text']))


#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

The word cloud can be saved and displayed using matplotlib and .show(). This is the result of all records, regardless of it being spam.


corporate
cds
stock
receipts
development
text
call
aurora
ve
news
dtype
wow
interest
david
visit
than
color
charges
study
irresistible
jim
money
fanny
gunslinger
printing
enron
request
add
update
object
special
made
trading
homes
software
research
please
naturally
Name
easy
case
unbelievable
Length

OUTPUT:


Problems
Output
Debug Console
Terminal
Ports
Code
PS C:\Users\Dave\Desktop\Technical Assessment 9> python -u "c:\Users\Dave\Desktop\Technical Assessment 9\TA9_Salvador.py"
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
Figure 1
aurora
irresistible
update
corporate
software
naturally
wanting
merrill
homes
show
but
additional
thanks
gpg
special
money
printing
stock
made
unbelievable
cds
identity
easy
forwarded
please
wow
enron
call
study
charges
compatibility
research
mode
invitation
fanny
gunslinger
development
this
muz
trading
go
Ln 58, Col 1
Spaces: 4
UTF-8
CRLF
{} Python

CODE:

```
# ========================================
# 1. IMPORTING DEPENDENCIES AND DATA
# ========================================

import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# Create dataframe from CSV
df = pd.read_csv('emails.csv')

df.head()

# ========================================
# 2. EXPLORATORY ANALYSIS
# ========================================

print("spam count: " + str(len(df.loc[df.spam == 1])))
print("not spam count: " + str(len(df.loc[df.spam == 0])))

print(df.shape)

df['spam'] = df['spam'].astype(int)

# Remove duplicate rows
df = df.drop_duplicates()

df = df.reset_index(inplace=False)[['text', 'spam']]

print(df.shape)

# ========================================
# 3. CLEANING THE TEXTS
# ========================================

clean_desc = []

for w in range(len(df.text)):

    desc = df['text'][w].lower()

    # Remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)
```

```
    # Remove tags
    desc = re.sub("&lt;/?.*?&gt;", " ", desc)

    # Remove digits and special characters
    desc = re.sub(r"(\d|\W)+", " ", desc)

    clean_desc.append(desc)

# Assign the cleaned descriptions to the dataframe
df['text'] = clean_desc

df.head(3)

# ========================================
# 4. REMOVING STOP WORDS
# ========================================

stop_words = [
    'is', 'you', 'your', 'and', 'the', 'to', 'from',
    'or', 'I', 'for', 'do', 'get', 'not', 'here',
    'in', 'im', 'have', 'on', 're', 'new', 'subject'
]

# Remove stop words from the text
df['text'] = df['text'].apply(
    lambda x: ' '.join(
        [word for word in x.split() if word not in stop_words]
    )
)

# ========================================
# 5. CONSTRUCT THE WORD CLOUD
# ========================================

wordcloud = WordCloud(
    width=800,
    height=800,
    background_color='black',
    stopwords=stop_words,
    max_words=1000,
    min_font_size=20
).generate(str(df['text']))

# Plot the word cloud
fig = plt.figure(figsize=(8, 8), facecolor=None)
```

```
plt.imshow(wordcloud)
plt.axis('off')
plt.show()

# Save the generated word cloud
wordcloud.to_file("spam_wordcloud.png")

print("Word Cloud successfully generated!")
```

CODE SCREENSHOT:

Welcome | TA9_Salvador.py

TA9_Salvador.py > ...

```
2   # ========================================
3   # 1. IMPORTING DEPENDENCIES AND DATA
4   # ========================================
5
6   import pandas as pd
7   import sqlite3
8   import regex as re
9   import matplotlib.pyplot as plt
10  from wordcloud import WordCloud
11
12
13  # Create dataframe from CSV
14  df = pd.read_csv('emails.csv')
15
16  df.head()
17
18
19  # ========================================
20  # 2. EXPLORATORY ANALYSIS
21  # ========================================
22
23  print("spam count: " + str(len(df.loc[df.spam == 1])))
24  print("not spam count: " + str(len(df.loc[df.spam == 0])))
25
26  print(df.shape)
27
28  df['spam'] = df['spam'].astype(int)
29
30  # Remove duplicate rows
31  df = df.drop_duplicates()
32
33  df = df.reset_index(inplace=False)[['text', 'spam']]
34
35  print(df.shape)
```

```
36
37
38  # ========================================
39  # 3. CLEANING THE TEXTS
40  # ========================================
41
42  clean_desc = []
43
44  for w in range(len(df.text)):
45
46      desc = df['text'][w].lower()
47
48      # Remove punctuation
49      desc = re.sub('[^a-zA-Z]', ' ', desc)
50
51      # Remove tags
52      desc = re.sub("&lt;/?.*?&gt;", " ", desc)
53
54      # Remove digits and special characters
55      desc = re.sub(r"(\d|\W)+", " ", desc)
56
57      clean_desc.append(desc)
58
59
60  # Assign the cleaned descriptions to the dataframe
61  df['text'] = clean_desc
62
63  df.head(3)
64
65
66  # ========================================
67  # 4. REMOVING STOP WORDS
68  # ========================================
```

```
69
70  stop_words = [
71      'is', 'you', 'your', 'and', 'the', 'to', 'from',
72      'or', 'I', 'for', 'do', 'get', 'not', 'here',
73      'in', 'im', 'have', 'on', 're', 'new', 'subject'
74  ]
75
76  # Remove stop words from the text
77  df['text'] = df['text'].apply(
78      lambda x: ' '.join(
79          [word for word in x.split() if word not in stop_words]
80      )
81  )
82
83
84  # ========================================
85  # 5. CONSTRUCT THE WORD CLOUD
86  # ========================================
87
88  wordcloud = WordCloud(
89      width=800,
90      height=800,
91      background_color='black',
92      stopwords=stop_words,
93      max_words=1000,
94      min_font_size=20
95  ).generate(str(df['text']))
96
```

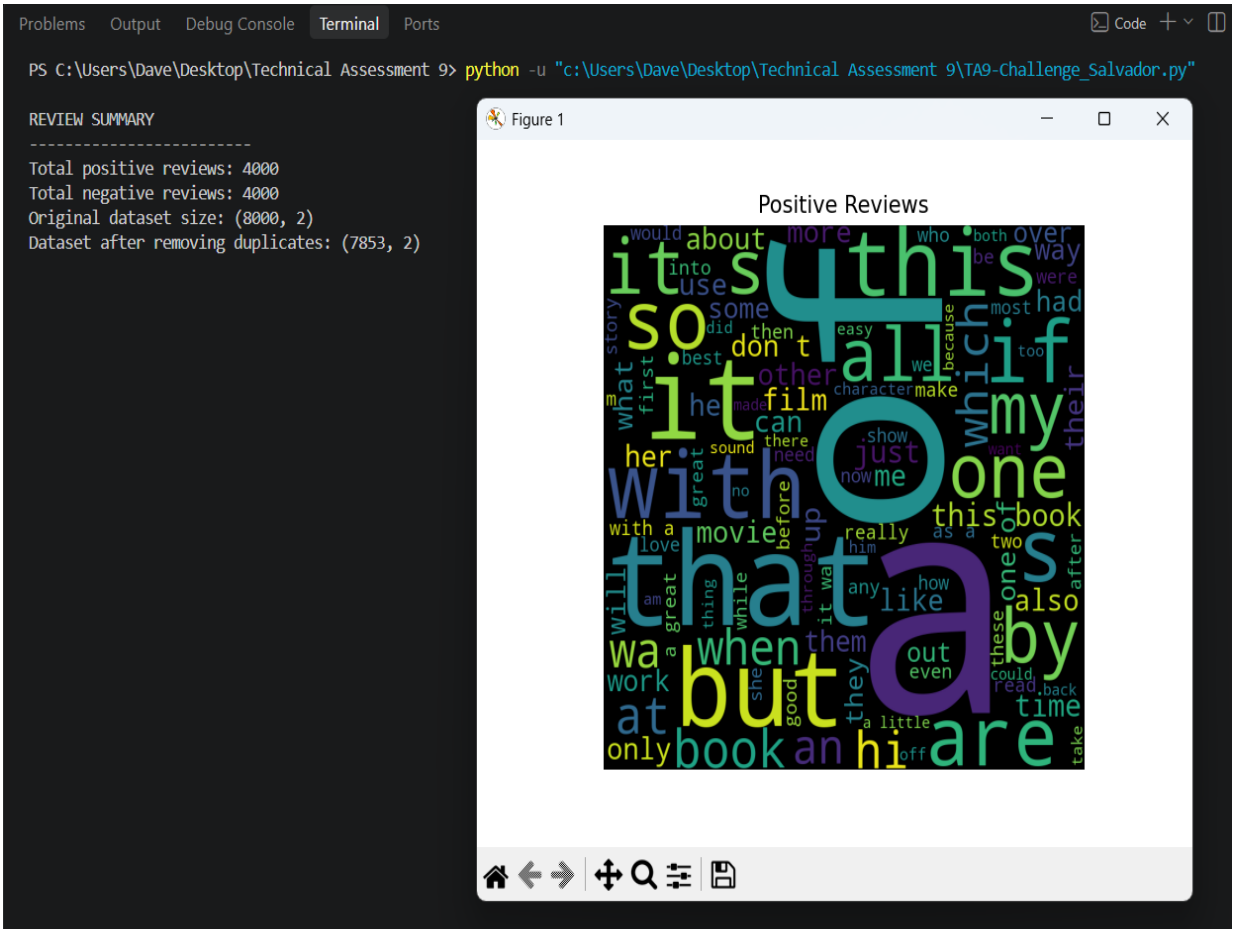
```
97
98   # Plot the word cloud
99   fig = plt.figure(figsize=(8, 8), facecolor=None)
100
101  plt.imshow(wordcloud)
102  plt.axis('off')
103  plt.show()
104
105
106  # Save the generated word cloud
107  wordcloud.to_file("spam_wordcloud.png")
108
109  print("Word Cloud successfully generated!")
110
```

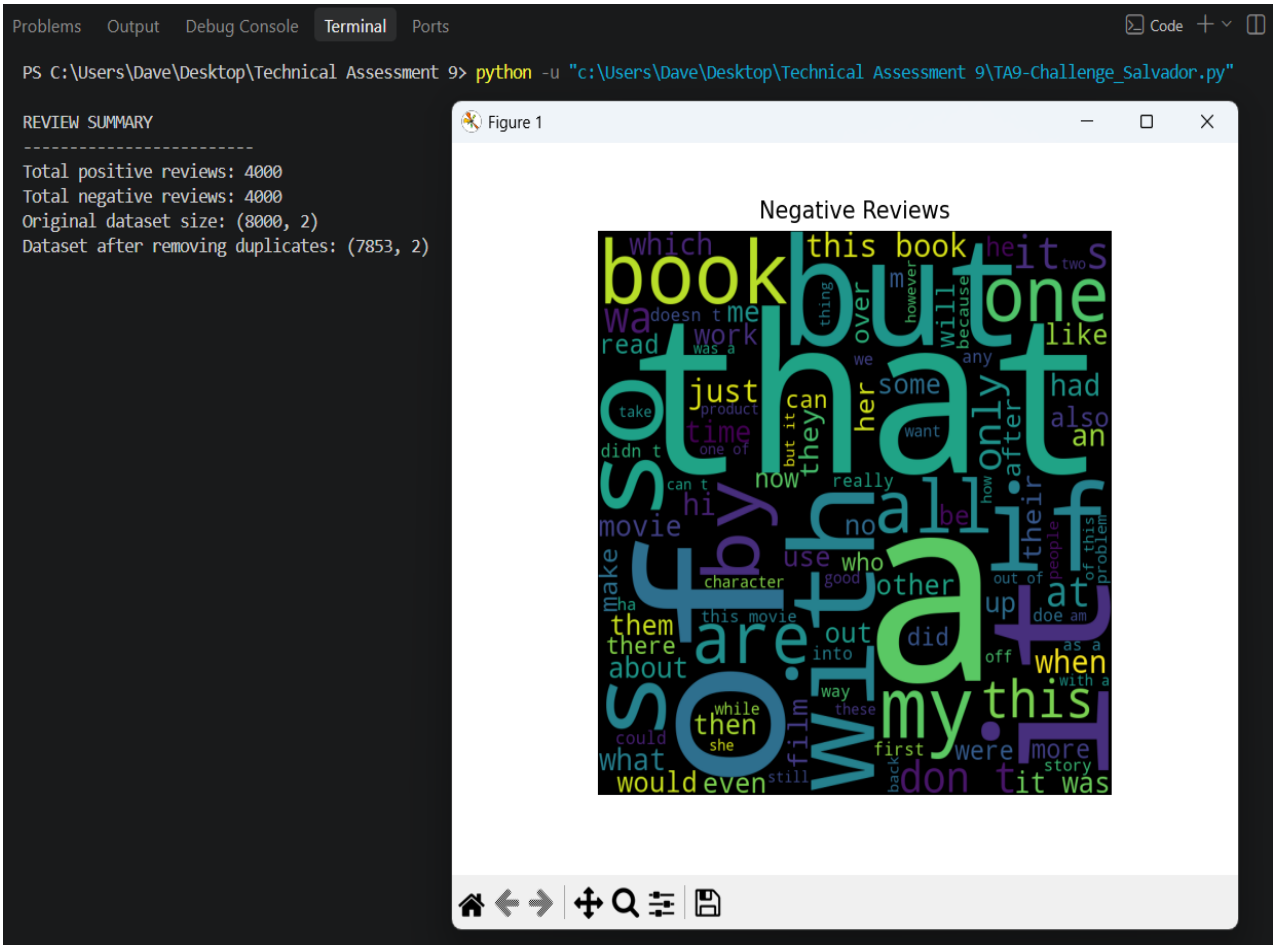
VII. CHALLENGE

- Repeat the procedure you did in Part VI., this time using any of the following open data sets found in this URL: https://analyticsindiamag.com/10-nlp-open-source-datasets-to-start-your-first-nlp-project/
- Clearly specify the intention of the output that you will generate.
- Compare and contrast sample outputs that you will generate.
- Provide a conclusion on whether you have achieved your goal or not.
- Provide a recommendation on things you did wrong and suggestions to improve and achieve the expected output.
- You may use existing Word Cloud generator like Wordle, Tagxedo, Tag Crowd, WordItOut, Word Cloud Generator for Chrome, Word Cloud Python tools or Google Word Cloud tools and check your output against the output of the generators. Analyze and explain results.

OUTPUT:




Problems
Output
Debug Console
Terminal
Ports
Code
PS C:\Users\Dave\Desktop\Technical Assessment 9> python -u "c:\Users\Dave\Desktop\Technical Assessment 9\TA9-Challenge_Salvador.py"
REVIEW SUMMARY
Total positive reviews: 4000
Total negative reviews: 4000
Original dataset size: (8000, 2)
Dataset after removing duplicates: (7853, 2)
Figure 1
Positive Reviews



CODE:

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from pathlib import Path

# Set the dataset directory
folder = Path("sorted_data_acl")

# Store all collected reviews
review_list = []

# Load positive and negative reviews
for path in folder.rglob("*.review"):

    review_type = path.stem
```

```
    if review_type not in ["positive", "negative"]:
        continue

    with open(path, "r", encoding="latin-1") as f:
        data = f.read()

    # Get the review text from the file
    extracted_reviews = re.findall(
        r"<review_text>(.*?)</review_text>",
        data,
        flags=re.DOTALL | re.IGNORECASE
    )

    for item in extracted_reviews:
        review_list.append({
            "text": item,
            "sentiment": review_type
        })

# Convert collected reviews into a DataFrame
df = pd.DataFrame(review_list)

# Display the number of reviews
print("\nREVIEW SUMMARY")
print("--------------------------")
print("Total positive reviews:", len(df[df["sentiment"] ==
"positive"]))
print("Total negative reviews:", len(df[df["sentiment"] ==
"negative"]))
print("Original dataset size:", df.shape)

# Remove duplicate reviews
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)

print("Dataset after removing duplicates:", df.shape)

# Text preprocessing
processed_reviews = []

for index in range(len(df)):

    text = df.loc[index, "text"].lower()

    # Remove punctuation and unnecessary symbols
```

```
        text = re.sub("[^a-zA-Z]", " ", text)

        # Remove HTML tags
        text = re.sub("&lt;/?.*?&gt;", " ", text)

        # Remove numbers and special characters
        text = re.sub(r"(\d|\W)+", " ", text)

        processed_reviews.append(text)

    # Update the DataFrame with cleaned text
    df["text"] = processed_reviews

    # List of stop words
    stop_words = [
        "is", "you", "your", "and", "the", "to", "from", "or",
        "i", "for", "do", "get", "not", "here", "in", "im",
        "have", "on", "re", "new", "subject"
    ]

    # Create a word cloud containing all reviews
    all_reviews = " ".join(df["text"])

    wordcloud = WordCloud(
        width=800,
        height=800,
        background_color="black",
        stopwords=stop_words,
        max_words=1000,
        min_font_size=20
    ).generate(all_reviews)

    # Show and save the overall word cloud
    plt.figure(figsize=(8, 8))
    plt.imshow(wordcloud)
    plt.axis("off")
    plt.savefig("challenge_wordcloud.png", bbox_inches="tight")
    plt.show()

    # Create individual word clouds for each sentiment
    sentiment_types = ["positive", "negative"]

    for review_type in sentiment_types:

        selected_reviews = " ".join(
            df.loc[df["sentiment"] == review_type, "text"]
```

```
)

if selected_reviews.strip():

    cloud = WordCloud(
        width=800,
        height=800,
        background_color="black",
        stopwords=stop_words,
        max_words=1000,
        min_font_size=20
    ).generate(selected_reviews)

    # Display the generated word cloud
    plt.figure(figsize=(8, 8))
    plt.imshow(cloud)
    plt.axis("off")
    plt.title(review_type.capitalize() + " Reviews")

    # Save the word cloud image
    plt.savefig(
        review_type + "_wordcloud.png",
        bbox_inches="tight"
    )

    plt.show()
```

CODE SCREENSHOT:

Welcome | TA9_Salvador.py | TA9-Challenge_Salvador.py

TA9-Challenge_Salvador.py > ...

```python
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from pathlib import Path

# Set the dataset directory
folder = Path("sorted_data_acl")

# Store all collected reviews
review_list = []

# Load positive and negative reviews
for path in folder.rglob("*.review"):

    review_type = path.stem

    if review_type not in ["positive", "negative"]:
        continue

    with open(path, "r", encoding="latin-1") as f:
        data = f.read()

    # Get the review text from the file
    extracted_reviews = re.findall(
        r"<review_text>(.*?)</review_text>",
        data,
        flags=re.DOTALL | re.IGNORECASE
    )

```


Welcome
TA9_Salvador.py
TA9-Challenge_Salvador.py
TA9-Challenge_Salvador.py > ...
flags=re.DOTALL | re.IGNORECASE
)
for item in extracted_reviews:
review_list.append({
"text": item,
"sentiment": review_type
})
Convert collected reviews into a DataFrame
df = pd.DataFrame(review_list)
Display the number of reviews
print("\nREVIEW SUMMARY")
print("------------------------")
print("Total positive reviews:", len(df[df["sentiment"] == "positive"]))
print("Total negative reviews:", len(df[df["sentiment"] == "negative"]))
print("Original dataset size:", df.shape)
Remove duplicate reviews
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)
print("Dataset after removing duplicates:", df.shape)
Text preprocessing
processed_reviews = []
for index in
Loading...
text = df.loc[index, "text"].lower()


Welcome
TA9_Salvador.py
TA9-Challenge_Salvador.py
TA9-Challenge_Salvador.py > ...
text = df.loc[index, 'text'].lower()
Remove punctuation and unnecessary symbols
text = re.sub("[^a-zA-Z]", " ", text)
Remove HTML tags
text = re.sub("</?.*?>", " ", text)
Remove numbers and special characters
text = re.sub(r"(\d|\W)+", " ", text)
processed_reviews.append(text)
Update the DataFrame with cleaned text
df["text"] = processed_reviews
List of stop words
stop_words = [
"is", "you", "your", "and", "the", "to", "from", "or",
"i", "for", "do", "get", "not", "here", "in", "im",
"have", "on", "re", "new", "subject"
]
Create a word cloud containing all reviews
all_reviews = " ".join(df["text"])
wordcloud = WordCloud(
width=800,
height=800,
background_color="black",
stopwords=stop_words,
max_words=1000,
min_font_size=20
).generate(all_reviews)

Welcome | TA9_Salvador.py | TA9-Challenge_Salvador.py

TA9-Challenge_Salvador.py > ...

```
93
94  # Show and save the overall word cloud
95  plt.figure(figsize=(8, 8))
96  plt.imshow(wordcloud)
97  plt.axis("off")
98  plt.savefig("challenge_wordcloud.png", bbox_inches="tight")
99  plt.show()
100
101 # Create individual word clouds for each sentiment
102 sentiment_types = ["positive", "negative"]
103
104 for review_type in sentiment_types:
105
106     selected_reviews = " ".join(
107         df.loc[df["sentiment"] == review_type, "text"]
108     )
109
110     if selected_reviews.strip():
111
112         cloud = WordCloud(
113             width=800,
114             height=800,
115             background_color="black",
116             stopwords=stop_words,
117             max_words=1000,
118             min_font_size=20
119         ).generate(selected_reviews)
120
121         # Display the generated word cloud
122         plt.figure(figsize=(8, 8))
123         plt.imshow(cloud)
124         plt.axis("off")
125         plt.title(review_type.capitalize() + " Reviews")
126
```

```
120
121         # Display the generated word cloud
122         plt.figure(figsize=(8, 8))
123         plt.imshow(cloud)
124         plt.axis("off")
125         plt.title(review_type.capitalize() + " Reviews")
126
127         # Save the word cloud image
128         plt.savefig(
129             review_type + "_wordcloud.png",
130             bbox_inches="tight"
131         )
132
133         plt.show()
134
```

1. Intention of the Output

The main goal of this activity is to generate Word Clouds using a dataset containing positive and negative customer reviews. I want to identify the most frequently used words and understand the common opinions expressed by customers. By generating three different Word Clouds, one for all reviews, one for positive reviews, and one for negative reviews, I can easily see the differences between the words used in each type of feedback. This also helps me understand how text analysis and data visualization can be used to organize and present information.

2. Compare and Contrast the Sample Outputs

When comparing the three outputs, the overall Word Cloud shows the most frequently used words from both positive and negative reviews. It gives a general idea of what customers usually talk about. On the other hand, the positive reviews Word Cloud focuses only on words from satisfied customers, while the negative reviews Word Cloud highlights words commonly used by customers who had a bad experience. The main difference is that the positive and negative Word Clouds allow me to understand each type of feedback separately, while the overall Word Cloud combines everything into one output. Larger words represent words that appear more frequently in the reviews.

3. Conclusion

I can say that I achieved the goal of this activity because my Python program successfully processed the dataset and generated three different Word Clouds. Through these outputs, I was able to visualize the commonly used words and compare positive and negative customer feedback. This activity also helped me understand the importance of cleaning the text before analyzing it because unnecessary words and characters can affect the results.

4. Recommendations and Suggestions for Improvement

One thing I would recommend improving in my code is the stop words list because some common words may still appear in the Word Clouds even though they do not provide much meaning. I can also improve the text cleaning process by removing unnecessary HTML characters, extra spaces, and other unwanted symbols. Another suggestion is to use online Word Cloud generators such as Wordle, Tagxedo, or WordItOut to compare their outputs with my Python-generated Word Clouds. This can help me check the results and explore other ways to present the data.

5. Analysis of the Results

Overall, the Python Word Cloud generator and online Word Cloud tools have the same purpose of showing frequently used words through different text sizes. However, I find Python more useful for this activity because I can control the dataset, clean the reviews, separate positive and negative feedback, and customize the appearance of the output. Through this activity, I learned how Word Clouds can make text data easier to understand and how they can help identify common opinions from a large collection of customer reviews.

VIII. REFERENCES

- Vahid Mirjalili(2017). Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow. India: Packt Publishing
- Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics

Online Sources:

- https://boostlabs.com/blog/what-are-word-clouds-value-simple-visualizations/
- https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb
- https://towardsdatascience.com/3-super-simple-projects-to-learn-natural-language-processing-using-python-8ef74c757cd9